

responsible for the physical layer of the Bluetooth architecture. It establishes and manages the physical radio frequency link between Bluetooth devices that forms a *piconet*.

Radio: This is a Bluetooth transceiver, which transmits data to and from the baseband. It is the lowest layer of the Bluetooth architecture. See Figure 6 for a complete view of the Bluetooth protocol stack.

Coding with Bluetooth

Before coding, you need the development files for linking to the BlueZ library. For this you have to install the *libbluetooth-dev* package:

```
sudo apt-get install libbluetooth-dev
```

Now, let's do some basic programming with Bluetooth like searching for a Bluetooth device.

Searching Bluetooth devices

To search for Bluetooth devices, you need some APIs. BlueZ exposes socket APIs that are similar to network socket programming. If you are familiar with network programming in Linux, learning the BlueZ APIs will be simple.

The basic data structure used to specify a 6-byte Bluetooth device address is *bdaddr_t*. In BlueZ, every Bluetooth address is stored as the *bdaddr_t* structure (see Figure 7).

BlueZ provides two functions to convert between the string and the *bdaddr_t* structure:

```
int str2ba( const char *str , bdaddr_t *ba );
int ba2str( const bdaddr_t *ba, char *str );
```

The header files required

Before using these APIs, you need some header files:

```
#include <bluetooth/bluetooth.h>
#include <bluetooth/hci.h>
#include <bluetooth/hci_lib.h>
```



Note: These headers files are available in the */usr/include/bluetooth* directory.

Getting the Bluetooth adapter ID

You can get a Bluetooth adapter resource number by using the following API:

```
int hci_get_route ( bdaddr_t *bdaddr );
```

Local Bluetooth adapters are assigned identifying numbers starting with 0, and a program must specify which adapter to use when allocating system resources. Usually, there is only one adapter or it does not matter which one is used, so passing NULL to *hci_get_route* will retrieve the ID

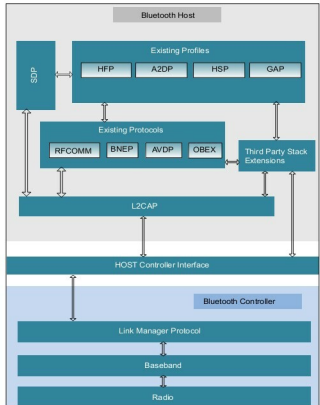


Figure 6: The complete Bluetooth protocol stack

```
/* BD Address */
typedef struct {
    uint8_t b[6];
}__attribute__((packed)) bdaddr_t;
```

Figure 7: Structure *bdaddr_t*

of the first available Bluetooth adapter:

```
dev_id = hci_get_route (NULL);
```



Note: If multiple adapters are present, then choose the adapter by passing the *char** representation of the address to *hci_dev_id*:

```
dev_id = hci_dev_id ( "01:43:24:9A:45:C3" );
```

Opening a file handle to the HCI

After getting the Bluetooth adapter ID, it's time to open a file handle to the host controller interface so that we can pass commands to the Bluetooth controller.

```
int fh = hci_open_dev(dev_id);
```